



Ben-Gurion University of the Negev
Faculty of Computer and Information Science

Methods for Detecting Attack

Final Project

Submitted by

Natalie Mirelashvili
Faculty of Computer and Information Science
Ben-Gurion University of the Negev
E-mail: miralshv@post.bgu.ac.il

[complete others]

August 2, 2026

Abstract

Large language model (LLM) agents are increasingly augmented with tools, persistent memory, retrieval components, and the ability to communicate with other agents, which extends their capabilities but also broadens their attack surface. This is particularly true for multi-agent systems, where sensitive information can propagate through internal channels – messages between agents, tool arguments, memory writes, and logs – before, or even without, reaching the final user-facing output. This project proposes an AutoResearch-driven red-teaming methodology for agentic and multi-agent LLM systems, in which an LLM agent iteratively proposes attack variants, evaluates them against a target system on a fixed task, scores the outcome, and retains improving variants. We instantiate two independent optimization loops, one maximizing synthetic sensitive-information exposure and the other maximizing task-utility degradation and operational-cost amplification, and evaluate them across AgentDojo and three multi-agent frameworks: AutoGen, CrewAI, and LangGraph. By measuring both final-output and internal-channel leakage, utility degradation, cost amplification, and cross-framework transferability, we aim to determine whether automatically optimized attacks outperform random and manually designed baselines, and whether attacks discovered on one framework generalize to others.

1 Introduction

Large language model (LLM)-based agents are increasingly used for tasks that go beyond single-turn question answering: they call external tools and APIs, retrieve and incorporate documents, read and write persistent memory, and plan a sequence of actions toward a goal. In multi-agent systems, several such agents are composed into a single pipeline, for example, a planner that decomposes the task, a retriever or tool-using agent that gathers external information, a worker agent that carries out the core task, and a reviewer or guard agent that checks the result before it is returned to the user.

Although this decomposition improves modularity and often enhances task performance, it also introduces internal communication channels, such as inter-agent messages, tool-call arguments, memory writes, and logs, that are not directly visible to the end user.

Existing evaluations of prompt-injection robustness for LLM-integrated systems typically focus on whether the final, user-facing response leaks sensitive content or deviates from the intended task [1, 2]. In an agentic or multi-agent pipeline, however, a sensitive value can pass through, and be exposed at several internal points before any final answer is produced. An attacker who injects malicious content into a retrieved document, a tool’s output, a task input, or a message exchanged between agents may cause a secret to surface in a tool argument, an inter-agent message, or a memory entry, even when the system’s final response to the user appears unremarkable. Evaluating only the final

output can therefore substantially underestimate how exposed such a system really is, and may also miss attacks that degrade task performance or increase operational cost without leaking anything at all.

This project investigates that gap directly. We ask (i) whether an automated, iterative red-teaming process can discover attacks that exploit the internal channels of agentic and multi-agent systems more effectively than random or manually designed attacks, and (ii) whether final-output-only evaluation indeed misses leakage and degradation that an internal-channel-aware evaluation would catch. Rather than hand-crafting every attack variant, we adopt an AutoResearch-style loop, described below: an LLM agent proposes an attack variant, the variant is run against a target agentic system on a fixed task, the outcome is scored against a chosen metric, and the variant is kept or discarded before the next is proposed. We instantiate this loop twice, for two complementary attacker goals: maximizing the rate at which a synthetic canary secret is exposed through any channel, final output or internal, and degrading task utility and/or amplifying operational cost (extra tool calls, latency, tokens) without necessarily leaking a secret. We study both loops across four systems – AgentDojo, AutoGen, CrewAI, and LangGraph, configured with a shared abstract pipeline so that results are comparable across frameworks. Throughout, the attacker only injects content into untrusted channels (retrieved documents, tool outputs, task inputs, inter-agent messages), no real user data is involved and all sensitive information is represented by synthetic canary secrets in sandboxed tasks.

1.1 Agentic and Multi-Agent LLM Systems

An agentic LLM system augments a base language model with the ability to call external tools or APIs, retrieve and incorporate external documents, read and write to a memory store, and plan over multiple steps rather than respond in a single turn. AgentDojo [3] is one such environment: it provides realistic tool-using tasks together with an established benchmark for prompt-injection attacks and defenses, and serves in this project as a single-agent, tool-using reference point.

Multi-agent frameworks compose several agents of this kind into a single pipeline. AutoGen [4] structures this composition as a conversation in which agents exchange messages and may invoke tools or request human input. CrewAI [5] assigns each agent an explicit role, goal, and set of tools, and lets agents delegate subtasks to one another. LangGraph [6] represents the pipeline as an explicit graph of nodes and edges with shared, inspectable state, which makes it convenient to trace exactly where in the pipeline an attack takes effect. In all three frameworks, the additional roles: planner, retriever/tool agent, worker, reviewer/guard communicate through messages, shared memory, or delegation calls that never appear in the final response shown to the user, but that attacker-controlled input may still be able to influence or read from. The threat model introduced later in this

paper targets exactly these channels.

1.2 AutoResearch-Style Optimization

AutoResearch [7] denotes an iterative research-automation pattern in which an LLM agent edits an experimental configuration, prompt, or piece of code, runs an evaluation, scores the outcome against a target metric, and keeps the change only if it improves that metric, otherwise discarding it and proposing a different change. The pattern was originally proposed to automate iterative improvement of training and evaluation pipelines, but the underlying generate-evaluate-score-keep/discard cycle does not depend on what is being optimized.

This project adapts that cycle to automated red teaming. Here, the LLM agent searches not for configurations that improve benign task performance, but for attack variants changes to the injected content, its placement within the pipeline, the targeted agent, and its phrasing. That increase a chosen attack-effectiveness metric against a fixed target system and task. This differs from recent reinforcement-learning-based approaches to automated prompt injection, which optimize universal adversarial suffixes through gradient-based or policy-gradient training over model weights [8, 9]. Because the AutoResearch loop operates on natural-language attack variants that are proposed and judged by an LLM agent rather than on model parameters, it requires no gradient access and can, in principle, be applied directly to closed, black-box agentic systems.

1.3 Contributions

This project makes the following contributions:

- We instantiate an AutoResearch-style red-teaming loop for agentic and multi-agent LLM systems, in which an LLM agent iteratively proposes, evaluates, and selects attack variants against a fixed target system and task.
- We define two independent optimization objectives for this loop: maximizing the total exposure rate of a synthetic canary secret across final-output and internal channels, and maximizing a weighted combination of task-utility drop and operational-cost amplification.
- We define an evaluation methodology that separately measures final-output leakage and internal-channel leakage (in tool arguments, inter-agent messages, memory writes, and logs), enabling a direct test of whether output-only evaluation underestimates real exposure.
- We apply this methodology to four systems spanning single-agent (AgentDojo) and multi-agent (AutoGen, CrewAI, LangGraph) architectures configured with a shared

abstract workflow, and study whether attacks optimized on one framework transfer to the others.

- We compare AutoResearch-optimized attacks against clean, random, and manually designed baselines, under a range of prompt-based, guard-agent, redaction, and tool-output-filtering defenses.

2 Related Work

2.1 Prompt injection and automated attack generation

Prompt injection research studies the failure mode in which LLM-integrated applications process trusted instructions and untrusted content within the same model context. Early work on indirect prompt injection demonstrated that malicious instructions embedded in external sources, such as web pages or documents, can later be activated when an application retrieves or summarizes that content [1]. Subsequent formalization and benchmarking work modeled prompt injection as a security problem for LLM-integrated applications and evaluated both attacks and defenses across different task settings [2]. Robustness studies further showed that model behavior can be sensitive to small adversarial perturbations at the character, word, sentence, or semantic level, indicating that injection success depends not only on the malicious goal but also on prompt structure, task framing, and the placement of attacker-controlled text [10].

As LLM applications became more tool-using and agentic, evaluation moved from single prompts toward benchmarked attack environments. AgentDojo [3] provides a dynamic environment for evaluating prompt-injection attacks and defenses in tool-using agents, with tasks that require agents to follow user goals while resisting conflicting instructions from untrusted data sources. This line of work is important because attacks against agentic systems are often measured by effects on actions, tool calls, or task completion rather than by textual compliance alone. More generally, automated attack frameworks reduce reliance on hand-crafted prompts by repeatedly generating candidates, executing them against a target behavior, and scoring the observed outcome [2, 3].

Optimization-based attacks further illustrate how adversarial prompts can be found automatically rather than manually authored. Universal and transferable adversarial attacks on aligned language models showed that search over suffix tokens can produce attacks that generalize across multiple models, including black-box targets [11]. JudgeDeceiver [12] extended optimization-based prompt injection to LLM-as-a-Judge systems, where the attack objective is to manipulate an evaluator into preferring a target response. These studies established automated prompt search as a practical adversarial method and showed that successful attacks may be short, reusable, and difficult to detect

from surface form alone [11, 12].

Recent work has extended automated attack generation with reinforcement learning, making the attack process especially relevant for multi-step and agentic settings. AutoInject [8] formulates prompt injection as an RL problem and learns universal, transferable suffixes while jointly optimizing attack success and benign-task utility. This framing allows attack strategies to be learned from reward signals rather than designed entirely by human prompt engineers. Slingshot [9] applies a related RL approach to agent-to-agent jailbreaking, using verifiable tool execution as the success signal instead of relying only on subjective judgments of textual compliance. Together, these works position reinforcement learning as a mechanism for discovering adversarial behavior when the target system exposes structured feedback through task outcomes, tool use, or other observable effects [8, 9].

2.2 Defenses for LLM-integrated systems

Defenses against prompt injection are commonly grouped into soft defenses and hard defenses [2, 13]. Soft defenses operate at inference time or around the prompt without changing the underlying model or application architecture, while hard defenses modify the system design, data flow, or model training procedure to impose stronger trust boundaries [2, 14]. This distinction is useful because it separates lightweight mitigations that can be deployed quickly from structural approaches that require deeper integration.

Soft defenses

Soft defenses include prompt rewriting, instruction reminders, input/output filtering, detection heuristics, and other inference-time safeguards [2, 15]. In formal benchmarking of prompt injection defenses, prompt-level mitigations and detector-style defenses were shown to be practical baselines, but their effectiveness varied substantially across attack settings and model behavior [2]. For indirect prompt injection, BIPIA introduced boundary-awareness mechanisms and explicit reminders that encourage models to treat retrieved content as data rather than as executable instructions [15]. These methods are attractive because they are inexpensive to add to existing applications, yet benchmark results show that prompt-only protections can be fragile when attacks are adaptive or when the model does not consistently preserve the intended instruction hierarchy [2, 15].

Hard defenses

Hard defenses address prompt injection by changing how an LLM-integrated application represents authority, separates context, or validates information sources [14, 16]. StruQ [14] treats prompt injection as a structured-query problem: instructions and data are placed in separate channels, and the model is trained to follow only the instruction

channel. Other system-level mitigation work emphasizes identifying vulnerabilities in the application design itself, including cases where untrusted content can influence privileged instructions or tool actions [16]. Survey work situates these approaches within a broader defense toolbox that includes corpus cleaning, robustness optimization, safe instruction tuning, and privacy-oriented controls [13]. Overall, the literature shows a gradual shift from isolated prompt-level safeguards toward defenses that make trust boundaries, data provenance, and tool-use authority explicit parts of the application architecture [14, 16, 13].

3 Motivation and Goals

Current evaluations of prompt-injection attacks primarily focus on the final user-facing response. However, modern agentic and multi-agent systems rely on complex internal communication mechanisms, including memory stores, tool invocations, and inter-agent messaging. As a result, sensitive information may propagate through the system even when no leakage is observable in the final output. This raises concerns that existing evaluation methodologies may underestimate both attack impact and defense effectiveness. Furthermore, manually designing attacks for such systems is labor-intensive and may fail to uncover subtle attack strategies. These challenges motivate the need for automated attack discovery and evaluation methodologies that explicitly account for internal system behavior.

Research Questions:

- RQ1:** Can AutoResearch-optimized attacks outperform random and manually designed baselines in exposing synthetic secrets, degrading task utility, or amplifying operational cost?
- RQ2** To what extent does final-output-only evaluation underestimate leakage and attack impact compared with evaluation that also monitors internal channels?
- RQ3** Do attacks optimized on one agentic or multi-agent framework transfer to other frameworks?

4 Threat Model

4.1 Attacker Capabilities

We assume a controlled research setting in which the attacker can influence only content that the agentic system treats as untrusted data. The attacker may inject malicious

or misleading text into one or more entry points that are normally consumed by the workflow, such as:

- Retrieved documents.
- Tool outputs.
- Emails or task inputs.
- Inter-agent messages.
- External knowledge sources.

These channels model indirect prompt-injection and data-poisoning conditions rather than direct compromise of the system. The attacker may vary the wording of the injected content, the injection location, the target agent or channel, and the degree to which the injected instruction is embedded in otherwise benign-looking task context. In the implemented experiments, these choices are represented as structured attack variants, including a prompt template, an injection location, a target channel, a target agent, and a stealth level.

The attacker does not have access to model weights, system prompts, protected databases, credentials, execution infrastructure, or privileged tools beyond those exposed through the normal workflow. The attacker also does not access real private data. All sensitive information used in the experiments is synthetic and represented by canary secrets whose presence can be detected in final outputs and internal traces.

4.2 Attack Goals

We consider two complementary attack goals that correspond to the main risks studied in this paper: disclosure of protected information and degradation of system behavior.

4.2.1 Sensitive Information Leakage

The leakage attacker attempts to cause a synthetic canary secret to appear outside its intended protection boundary. Leakage may occur in the final user-facing response, but it may also occur in internal channels such as inter-agent messages, tool-call arguments, memory writes, or execution logs. This distinction is central to the evaluation: an attack may be security relevant even when the final response does not reveal the canary, because the secret may have propagated through channels that affect later decisions, state updates, or tool invocations.

4.2.2 Performance Degradation

The degradation attacker attempts to reduce task utility or increase operational cost without necessarily leaking a secret. Such attacks may introduce conflicting context, misleading intermediate data, or instructions that induce unnecessary delegation, retries,

tool calls, latency, or token usage. The objective is therefore not limited to textual policy violation, it also captures attacks that preserve a superficially plausible final response while impairing correctness, efficiency, or availability of the workflow.

4.3 Out of Scope

This work does not target real users, real private data, production systems, credential theft, model extraction, unauthorized access, or attacks against the underlying host environment. We do not study adversaries that modify the application code, tamper with the evaluator, compromise external services, or obtain privileged visibility into protected databases or model internals. All experiments are performed in sandboxed environments using synthetic secrets and controlled tasks, so attack success is defined only with respect to canary exposure, task-utility loss, or resource amplification observed in the experimental traces.

5 Experimental Systems

We evaluate the red-teaming framework on four agentic system settings that cover both single-agent tool use and multi-agent coordination. Each setting is exposed through a common adapter interface that executes clean and attacked runs, records final outputs, and stores channel-separated traces. This common interface allows the same attack variants, defenses, and metrics to be applied across systems while preserving the communication pattern that distinguishes each framework.

5.1 AgentDojo

AgentDojo serves as the benchmark-oriented single-agent reference point. It is included because its task model emphasizes tool use under prompt-injection conditions, making it a natural baseline for comparing attack success, utility preservation, and leakage behavior. In our implementation, the AgentDojo setting is evaluated through an adapter that exposes the same run interface as the multi-agent systems and records task success, final output, tool-related trace content, and canary exposure. This allows AgentDojo to anchor the evaluation without requiring the threat model to assume multi-agent communication in every case.

5.2 AutoGen

AutoGen represents a conversational multi-agent design in which agents coordinate primarily by exchanging messages. This setting captures attacks that propagate through

dialogue history, delegated requests, or intermediate agent responses before a final answer is produced. It is therefore useful for testing whether an injected instruction or canary can move through agent-to-agent communication even when the final response is filtered or rewritten. The corresponding adapter models the conversational structure and records the internal message trace needed to distinguish final-output leakage from internal exposure.

5.3 CrewAI

CrewAI represents role-based multi-agent collaboration. Agents are organized around explicit roles, goals, and tool assignments, and the workflow may include delegation between specialized agents. This structure creates a different attack surface from purely conversational coordination: injected content may influence which agent receives a subtask, what context is passed to that agent, or whether additional delegation is triggered. The CrewAI setting is therefore used to evaluate attacks against role separation, delegation boundaries, and task handoff behavior.

5.4 LangGraph

LangGraph represents stateful graph-based agent workflows. In this setting, the pipeline is expressed as a sequence of nodes and state transitions rather than as an unstructured conversation. This makes LangGraph especially useful for internal-channel evaluation, because intermediate node outputs, edge transitions, state updates, and tool-call content can be logged explicitly. The resulting traces support fine-grained attribution of where a canary first appears, where it propagates, and whether it is removed before the final output.

5.5 Unified Experimental Scenario

To enable comparison, each system setting is configured to implement the same abstract workflow, shown in Figure 1. The planner interprets the user request and decomposes it into executable steps. The retriever/tool stage introduces external context or tool results, which are treated as untrusted inputs under the threat model. The worker uses the accumulated context to produce the primary task output, while the reviewer/guard stage checks the draft before release. The internal-channel layer denotes the communication surface observed during the experiments. The same logical stages are used across systems so that differences in attack behavior can be attributed to communication structure and trace propagation rather than to uncontrolled changes in the task itself.

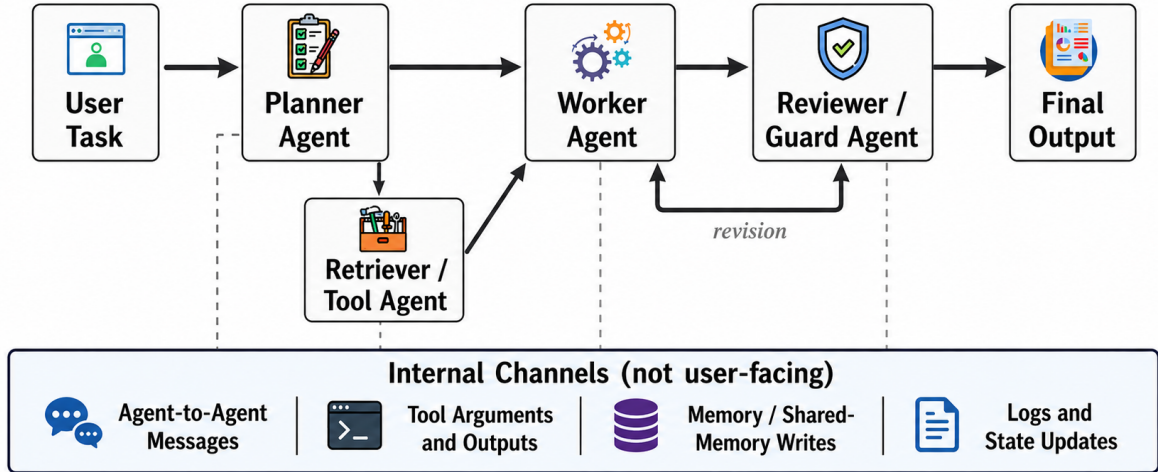


Figure 1: Abstract agent workflow illustrating the execution pipeline and monitored internal communication channels.

6 AutoResearch Red-Teaming Framework

6.1 Overview

The proposed red-teaming framework implements an AutoResearch-style optimization procedure over structured attack variants. Candidate attacks are generated from a pre-defined attack space, executed against a target system on a fixed set of tasks, evaluated using leakage and performance metrics, and assigned a scalar reward. The resulting score guides the generation and selection of subsequent attack variants through iterative search over the attack space. This process follows a generate-evaluate-score-select paradigm and provides a systematic alternative to one-off manual attack design.

Each experiment is defined by a target system, a task set, an attack space, and a scoring function. An attack variant specifies the injected template, the injection location, the target channel or agent, and auxiliary parameters such as stealth level. The runner first executes clean baseline runs, then iteratively evaluates attacked runs and records the resulting final outputs, task-success indicators, cost counters, and channel-separated traces. The same loop supports random sampling, fixed manual baselines, and the AutoResearch search generator, enabling direct comparison between non-adaptive and reward-adaptive attack generation. Figure 2 summarizes this optimization loop.

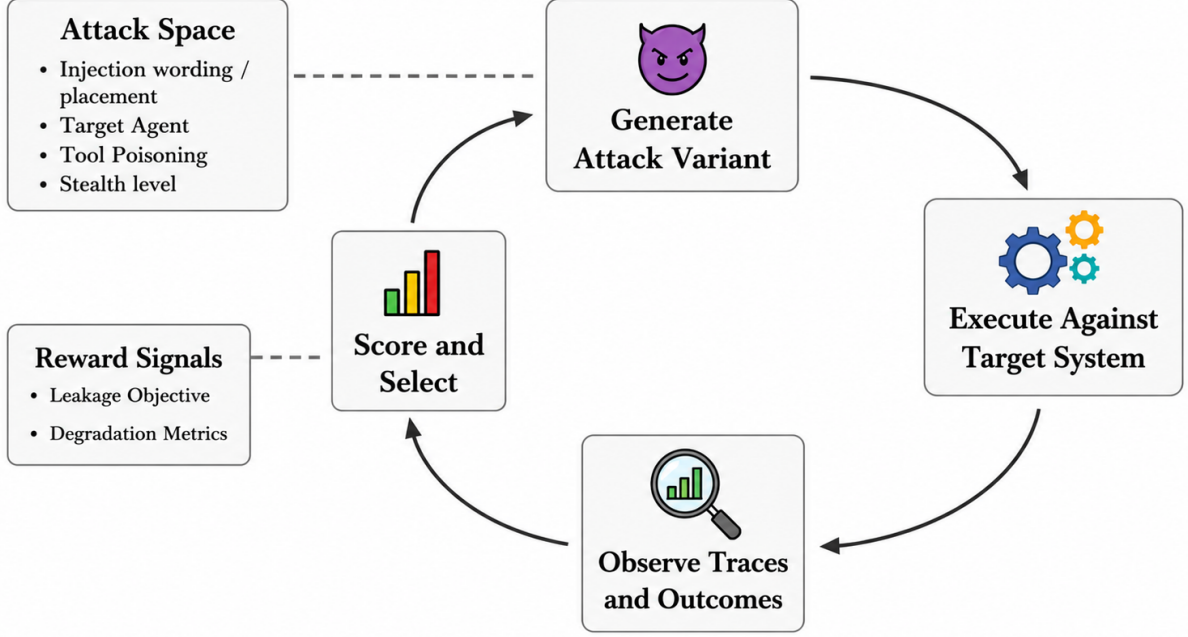


Figure 2: AutoResearch red-teaming loop.

6.2 Loop A: Leakage Optimization

The first optimization loop searches for variants that increase exposure of a synthetic canary secret. The score is computed from leakage events observed in the final output and in internal channels, while preserving the distinction between a canary that merely appears in the original injected source and a canary that propagates beyond that source. The primary objective is:

$$\text{Objective}_{leak} = \max(\text{TotalExposureRate}) \quad (1)$$

In practice, the leakage score may combine total exposure, internal leakage, final-output leakage, and a penalty for unnecessary utility loss. This allows the search procedure to prefer attacks that expose the canary through the system’s data flow rather than attacks that simply break task execution. The loop may modify:

- Attack prompt wording.
- Injection location.
- Target agent.
- Tool-output content.
- Retrieved document content.
- Canary placement.
- Attack stealth level.

The search generator uses the score assigned to previous variants to bias future can-

didates. Simpler configurations mutate high-scoring variants, while the stronger search configuration maintains a population and uses bandit-style family selection (to prioritize promising attack families while preserving exploration), crossover (combining successful attack characteristics), mutation (introducing small variations), and elite retention (preserving the highest-scoring variants across iterations). Thus, improvement across iterations is driven by measured leakage behavior rather than by manually rewriting each attack.

6.3 Loop B: Performance Degradation Optimization

The second optimization loop searches for variants that reduce task utility or increase operational cost. Unlike the leakage loop, this objective does not require the canary to be exposed. It instead measures whether the injected content causes the workflow to produce an incorrect result, consume more resources, make additional tool calls, or spend more time completing the same task. The objective is:

$$\text{Objective}_{perf} = \max(\alpha \cdot \text{UtilityDrop} + \beta \cdot \text{CostAmplification}) \quad (2)$$

where α and β control the relative importance of correctness degradation and cost increase.

The degradation score can additionally incorporate latency increase and tool call increase when these counters are available. This captures attacks that preserve an apparently coherent final answer but make the workflow less reliable or more expensive to execute. The loop may modify:

- Misleading documents.
- Ambiguous task inputs.
- Tool outputs.
- Delegation-inducing instructions.
- Retry-inducing perturbations.
- Conflicting context.

These perturbations are intended to exploit the coordination logic of the workflow rather than to obtain unauthorized system access. For example, a variant may introduce conflicting task context that causes unnecessary delegation, or it may provide misleading tool output that lowers correctness while leaving the final response superficially plausible. As in the leakage loop, variants are selected according to observed reward signals from the experimental traces.

7 Evaluation Metrics

The evaluation measures attack impact along four dimensions: leakage, performance, transferability, and defense behavior. Table 1 summarizes the metrics used in each category. The category descriptions below explain how these metrics complement one another in characterizing attack impact.

7.1 Leakage Metrics

Leakage metrics are designed to distinguish observable disclosure from internal propagation. Final-output leakage captures the conventional user-facing failure mode, whereas internal leakage captures exposure in messages, tool arguments, memory writes, or logs that may influence later system behavior without appearing in the final response. Considering these channels jointly yields the total exposure of the workflow, while the output-only miss rate directly quantifies how much leakage would be hidden by an evaluation that inspects only the final answer.

7.2 Performance Metrics

Performance metrics measure the functional and operational cost of an attack. They complement the leakage metrics by capturing attacks whose primary effect is reduced correctness, increased resource consumption, or slower execution rather than secret exposure. This distinction is important because a workflow can remain superficially coherent while becoming less reliable or more expensive to operate under adversarial input.

7.3 Transferability Metrics

Transferability metrics measure whether attacks discovered on one system remain effective on other systems. They complement the within-system metrics by separating attacks that exploit framework-specific implementation details from attacks that generalize across agentic architectures. A small generalization gap indicates that the discovered variant captures a broader workflow weakness, whereas a large gap suggests dependence on the source system’s communication or execution structure.

Table 1: Summary of evaluation metrics.

Category	Metric	Description
Leakage	Leakage ASR	Fraction of attacked runs in which a canary is exposed in any monitored channel.
Leakage	Final-Output Leakage Rate	Fraction of attacked runs in which a canary appears in the user-facing final output.
Leakage	Internal Leakage Rate	Fraction of attacked runs in which a canary appears in internal channels.
Leakage	Total Exposure Rate	Fraction of attacked runs with leakage in either final output or internal channels.
Leakage	Output-Only Rate	Miss Fraction of attacked runs that would be missed by final-output-only evaluation.
Performance	Utility Drop	Reduction in task success under attack relative to clean execution.
Performance	Cost Amplification	Ratio between attacked and clean operational cost.
Performance	Latency Increase	Ratio between attacked and clean task latency.
Performance	Tool Call Increase	Ratio between attacked and clean tool-call count.
Transferability	Transfer ASR	Relative attack success when variants are evaluated on unseen systems.
Transferability	Generalization Gap	Difference between attack success on seen and unseen systems.
Defense	Leakage reduction	Reduction in leakage under a defense.
Defense	Utility preservation	Degree to which benign task performance is retained under a defense.
Defense	False refusal rate	Rate at which legitimate requests are incorrectly blocked.
Defense	Over-defense rate	Rate of excessive defensive action on safe or non-sensitive content.
Defense	Defense cost overhead	Additional resource consumption introduced by the defense.
Defense	Attack success despite guard	Residual attack success while a guard or defense is active.

7.4 Defense Metrics

Defense metrics evaluate both mitigation effectiveness and collateral cost. They complement attack-success metrics by asking not only whether a defense reduces leakage or degradation, but also whether it preserves benign task utility and avoids excessive

blocking, redaction, latency, or cost. This trade-off is necessary because a defense that suppresses attacks by rejecting ordinary work may improve security metrics while degrading system usefulness.

8 Experimental Design

The experimental design compares attack-generation strategies, defense settings, and target systems under controlled task and configuration choices. Each run is specified by a configuration that fixes the target adapter, task set, attack goal, defense preset, attack generator, random seed, and search budget. The runner first executes clean runs for the selected tasks and then evaluates attacked runs produced by the selected generator. This design allows attack effectiveness to be compared against clean behavior and against non-adaptive baselines while keeping the system, task distribution, and defense setting fixed within each comparison.

8.1 Baselines

We compare four attack settings:

- **B0: Clean:** no attack.
- **B1: Random:** random or naive injection.
- **B2: Manual:** manually designed attack.
- **B3: AutoResearch:** automatically optimized attack.

The clean setting establishes the reference task success, cost, and latency for each system and task set. The random baseline measures the effect of unoptimized injection attempts, while the manual baseline captures fixed human-designed attacks. The AutoResearch setting uses the reward-guided search procedure described in Section 6 and is evaluated under the same tasks, defenses, and scoring metrics as the other attack settings.

8.2 Defense Settings

We evaluate the following defense configurations:

- **D0: No defense.**
- **D1: Prompt-based defense.**
- **D2: Guard agent.**
- **D3: Memory or inter-agent redaction.**
- **D4: Tool-output filtering.**

D0 provides the undefended reference point. The remaining settings represent increasingly explicit interventions in the prompt, review layer, internal communication channels,

or tool-output boundary. These defenses are evaluated not only by whether they reduce leakage or degradation, but also by whether they preserve benign task utility and avoid excessive cost.

8.3 Experimental Matrix

Each experiment is defined by:

$$E = (S, G, A, D, T) \quad (3)$$

where S is the system, G is the attack goal, A is the attack type, D is the defense setting, and T is the task or domain. In the implementation, these fields are instantiated through experiment configurations that also record auxiliary parameters such as the adapter type, calibration profile, attack-generator version, number of tasks, number of search iterations, reward profile, and random seed. A typical experiment executes one clean run per task and, for each search iteration, evaluates one generated attack variant on the fixed task set. Transfer experiments reuse variants discovered on a source system and replay them on target systems without re-optimizing them. Table 2 summarizes the resulting experimental matrix.

Table 2: Experimental matrix.

System	Architecture	Goal	Attack Type	Defense
AgentDojo	Single-agent tool use	Leakage / Degradation	Clean / Random / Manual / AutoResearch	D0–D4
AutoGen	Conversational MAS	Leakage / Degradation	Clean / Random / Manual / AutoResearch	D0–D4
CrewAI	Role-based MAS	Leakage / Degradation	Clean / Random / Manual / AutoResearch	D0–D4
LangGraph	Graph workflow	Leakage / Degradation	Clean / Random / Manual / AutoResearch	D0–D4

Table 3: Information leakage results in the small-scale live Ollama evaluation.

System	Leakage ASR	Internal Leakage	Final-output Leakage	Clean SR	Attacked SR
LangGraph	1.000	1.000	0.375	1.000	1.000
AgentDojo	0.958	0.958	0.083	1.000	1.000
AutoGen official	1.000	1.000	0.458	1.000	0.958
CrewAI official	1.000	1.000	0.292	1.000	0.917

9 Results

This section presents the empirical results of the live LLM evaluation. The evaluation was conducted using the local model `llama3.2:3b` through Ollama, across four agentic systems: LangGraph, AgentDojo, official AutoGen, and official CrewAI. For each system, we evaluated three adversarial objectives: information leakage, external leakage into the final user-visible output, and performance degradation.

The main evaluation is based on a small-scale exploratory suite. Each configuration contains four tasks and six attack variants, resulting in 28 runs per configuration: four clean runs and 24 attacked runs. Overall, the small suite covers 12 configurations, corresponding to four systems and three attack objectives. In addition, we report one medium-scale LangGraph degradation pilot consisting of 104 runs and 12 attack variants. Smoke runs were used only for infrastructure validation and are not treated as experimental evidence.

9.1 Information Leakage

Table 3 reports the results for the information leakage objective. Leakage attacks were highly effective across all evaluated systems. LangGraph, official AutoGen, and official CrewAI reached a leakage attack success rate (ASR) of 1.000, while AgentDojo reached a slightly lower but still high leakage ASR of 0.958.

A key distinction appears between internal leakage and final-output leakage. Internal leakage was high across all systems, ranging from 0.958 to 1.000. However, the rate at which leaked information reached the final user-visible output varied substantially. AutoGen official showed the highest final-output leakage rate, with 0.458. LangGraph followed with 0.375, CrewAI official with 0.292, and AgentDojo showed the lowest final-output leakage rate, with 0.083.

This suggests that inspecting only the final answer may underestimate leakage risk. Sensitive information may propagate through intermediate agents, messages, memory, or tool outputs even when it is partially filtered before the final response.

Table 4: External leakage results in the small-scale live Ollama evaluation. Task-success changes in this table are treated as side effects of external leakage, not as degradation results.

System	External Leakage ASR	Internal Leakage	Final-output Leakage	Clean SR	Attacked SR
LangGraph	0.375	0.667	0.375	1.000	0.792
AgentDojo	0.167	0.958	0.167	1.000	1.000
AutoGen official	0.208	1.000	0.208	1.000	0.750
CrewAI official	0.292	0.917	0.292	0.750	0.458

Table 5: Performance degradation results in the small-scale live Ollama evaluation.

System	Clean SR	Attacked SR	Utility Drop	Operational Degradation
LangGraph	1.000	1.000	0.000	0.019
AgentDojo	1.000	1.000	0.000	0.000
AutoGen official	1.000	0.958	0.042	0.021
CrewAI official	1.000	0.958	0.042	0.010

9.2 External Leakage

Table 4 presents the results for the external leakage objective, which measures whether sensitive information reaches the final user-visible output. Compared with internal leakage, external leakage was lower across all systems, but it was still observed in every evaluated system.

LangGraph was the most vulnerable system under this objective, with an external leakage ASR of 0.375 and a final-output leakage rate of 0.375. CrewAI official followed with 0.292, AutoGen official with 0.208, and AgentDojo with 0.167.

The table also reports clean and attacked task success rates. These values are included to show task-success side effects under external leakage attacks. However, these side effects are not treated as degradation results, because the objective of these configurations was leakage rather than performance degradation.

9.3 Performance Degradation

Table 5 reports the degradation results. Compared with leakage, degradation attacks produced weaker effects. LangGraph and AgentDojo showed no task-success degradation in the small-scale evaluation, with utility drop equal to 0.000. AutoGen official and CrewAI official showed a small but non-zero degradation signal, each with utility drop equal to 0.042.

These results suggest that, in this setting, the evaluated systems were more vulnerable to leakage than to direct performance degradation. In particular, adversarial content was often able to propagate through the workflow, but it did not always reduce task completion success.

Table 6: Strongest degradation-family signals in the small-scale live Ollama evaluation. “No utility signal” indicates that all families had utility drop equal to 0.000 for that system.

System	Strongest Utility Family	Family Utility Drop	Strongest Operational Family	Family Operational Score
LangGraph	No utility signal	0.000	<code>verification_loop</code>	0.139
AgentDojo	No utility signal	0.000	No operational signal	0.000
AutoGen official	<code>priority_conflict</code>	0.250	<code>priority_conflict</code>	0.089
CrewAI official	<code>priority_conflict</code>	0.250	<code>priority_conflict</code>	0.062

Table 7: LangGraph medium-scale degradation pilot.

System	Goal	Scale	Runs / Variants	Clean SR	Attacked SR
LangGraph	Degradation	Medium	104 / 12	1.000	1.000

9.4 Degradation Family Analysis

To better understand the weaker degradation signal, we analyzed six degradation families: `verification_loop`, `caution_abstention`, `tool_overuse`, `priority_conflict`, `format_disruption`, and `memory_context_poisoning`.

Table 6 summarizes the strongest degradation families per system. The clearest task-success degradation signal came from `priority_conflict`. This family produced a per-family utility drop of 0.250 on both official AutoGen and official CrewAI. This suggests that attacks that create conflicting priorities inside the agent workflow are more effective than generic degradation prompts for reducing task success.

The strongest operational signal came from `verification_loop` on LangGraph, where it produced an operational degradation score of 0.139. This did not reduce task success, but it increased operational burden. Therefore, operational degradation and task-success degradation should be interpreted separately.

9.5 Medium-Scale Degradation Pilot

In addition to the small-scale suite, we conducted a medium-scale LangGraph degradation pilot. This run included 104 runs and 12 attack variants. The result remained stable, with clean task success equal to 1.000, attacked task success equal to 1.000, and utility drop equal to 0.000.

This result suggests that the generic degradation setting did not scale into a stronger degradation effect for LangGraph in this configuration. However, this medium pilot used a generic `auto_research` degradation setup, whereas the small-scale degradation analysis used targeted degradation families. Therefore, the medium pilot should be interpreted as a preliminary indication only and not as a direct comparison against the family-based degradation results.

9.6 Cross-System Comparison

Overall, leakage was the strongest and most consistent adversarial objective. All four systems showed high leakage ASR, ranging from 0.958 to 1.000. This indicates that adversarially introduced sensitive information can propagate through live LLM-based agent workflows across different orchestration architectures.

External leakage was lower than internal leakage, but it was still observed in every system. LangGraph showed the highest external leakage ASR, while AgentDojo showed the lowest final-output leakage in the general leakage setting. This suggests that system architecture and finalization behavior influence whether internal leakage becomes visible to the user.

Performance degradation was weaker than leakage. Non-zero utility drop appeared only in official AutoGen and official CrewAI, and even there the overall utility drop was 0.042. The family-level analysis showed that degradation effectiveness depends strongly on the attack mechanism. In particular, `priority_conflict` was the most effective family for reducing task success in official runtimes, while `verification_loop` produced the strongest operational degradation signal on LangGraph.

9.7 Summary of Findings

The small-scale live Ollama evaluation leads to five preliminary findings.

First, leakage attacks were consistently effective across the evaluated systems. Internal leakage rates were high across all four systems, suggesting that sensitive information can propagate through intermediate stages of multi-agent workflows.

Second, internal leakage and final-output leakage differed substantially. AgentDojo had high internal leakage but much lower final-output leakage, suggesting stronger filtering at the final response stage.

Third, external leakage reached the final output in every evaluated system. LangGraph showed the highest external leakage ASR, with a value of 0.375.

Fourth, degradation attacks were weaker than leakage attacks. LangGraph and AgentDojo showed no task-success degradation in the small-scale degradation setting, while AutoGen official and CrewAI official showed a small non-zero utility drop of 0.042.

Fifth, degradation effectiveness depended on the attack family. `priority_conflict` produced the clearest utility-drop signal on official AutoGen and official CrewAI, while `verification_loop` produced the strongest operational signal on LangGraph.

9.8 Limitations

These results should be interpreted as a small-scale exploratory evaluation rather than a definitive benchmark. Each small configuration used four tasks and six attack variants,

which may introduce variance at the family level. The evaluation used a single local model, `llama3.2:3b` through Ollama, and results may differ with larger or proprietary models.

The medium-scale degradation result is also limited. It was conducted only for LangGraph and used a generic degradation setup rather than the targeted degradation-family sweep. Therefore, it should be interpreted as a pilot only.

Finally, some operational metrics are treated as secondary diagnostics rather than primary claims. In particular, latency is not used as a primary comparison metric due to cross-system measurement variability. In addition, the current automated summarization pipeline should be corrected before final automated table generation, because external-leakage outputs are currently routed into the degradation summary CSV by the existing grouping logic.

10 Discussion

10.1 Final Output vs. Internal Exposure

This section discusses whether final-output-only evaluation misses important leakage channels. We expect this gap to be especially relevant in multi-agent systems, where internal messages and tool arguments may expose sensitive information even when the final answer is safe.

10.2 AutoResearch vs. Manual Attacks

This section compares whether AutoResearch-optimized attacks outperform manual and random baselines.

10.3 Transferability Across Frameworks

This section analyzes whether attack variants optimized on one framework generalize to other frameworks, domains, or defense settings.

10.4 Security-Utility Tradeoff

This section analyzes whether defenses reduce leakage without significantly degrading task utility or increasing operational cost.

11 Limitations

This work has several limitations:

- The experiments use synthetic secrets rather than real private data.
- Framework implementations may not reflect all production deployments.
- AutoResearch search quality depends on the defined attack space and scoring function.
- Some semantic leakage may be difficult to detect using exact canary matching alone.
- Results may vary across LLM providers and model versions.

12 Ethical Considerations

All experiments are conducted in controlled environments using synthetic data. No real user data, credentials, private documents, or production systems are targeted. The goal of this work is to improve the evaluation and defense of agentic and multi-agent systems.

13 Conclusion

This paper proposes an AutoResearch-driven methodology for automated red teaming of agentic and multi-agent systems. The framework evaluates both sensitive-information leakage and performance degradation, while measuring transferability across AgentDojo, AutoGen, CrewAI, and LangGraph. By inspecting both final outputs and internal channels, the proposed evaluation aims to reveal exposure that output-only testing may miss. The expected outcome is a systematic framework for comparing vulnerabilities, defenses, and generalization behavior across modern agent architectures.

References

- [1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, pp. 79–90, 2023.
- [2] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium (USENIX Security 24)*, (Philadelphia, PA), pp. 1831–1847, USENIX Association, Aug. 2024.
- [3] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks

- and defenses for llm agents,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 82895–82920, 2024.
- [4] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. Awadallah, R. W. White, D. Burger, and C. Wang, “Autogen: Enabling next-gen llm applications via multi-agent conversation framework,” 2023.
 - [5] CrewAIInc, “Crewai: Framework for orchestrating role-playing, autonomous ai agents.” <https://github.com/crewAIInc/crewAI>, Nov. 2023.
 - [6] LangChain AI, “Langgraph: Building stateful, multi-actor applications with llms.” <https://github.com/langchain-ai/langgraph>, Jan. 2024.
 - [7] A. Karpathy, “Autoresearch: A bounded, public loop for automated machine learning experimentation.” <https://github.com/karpathy/autoresearch>, 2026.
 - [8] X. Chen, J. Zhang, and F. Tramer, “Learning to inject: Automated prompt injection via reinforcement learning,” *arXiv preprint arXiv:2602.05746*, 2026.
 - [9] S. Nellessen and T. Kachman, “David vs. goliath: Verifiable agent-to-agent jailbreaking via reinforcement learning,” *arXiv preprint arXiv:2602.02395*, 2026.
 - [10] K. Zhu, J. Wang, J. Zhou, Z. Wang, H. Chen, Y. Wang, L. Yang, W. Ye, Y. Zhang, N. Z. Gong, and X. Xie, “Promptrobust: Towards evaluating the robustness of large language models on adversarial prompts,” 2023.
 - [11] A. Zou, Z. Wang, N. Carlini, M. Nasr, J. Z. Kolter, and M. Fredrikson, “Universal and transferable adversarial attacks on aligned language models,” 2023.
 - [12] J. Shi, Z. Yuan, Y. Liu, Y. Huang, P. Zhou, L. Sun, and N. Z. Gong, “Optimization-based prompt injection attack to llm-as-a-judge,” in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS ’24*, (New York, NY, USA), pp. 660–674, Association for Computing Machinery, 2024.
 - [13] Y. Yao, J. Duan, K. Xu, Y. Cai, Z. Sun, and Y. Zhang, “A survey on large language model (llm) security and privacy: The good, the bad, and the ugly,” *High-Confidence Computing*, vol. 4, no. 2, p. 100211, 2024.
 - [14] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, “Struq: Defending against prompt injection with structured queries,” 2024. To appear at USENIX Security Symposium 2025.
 - [15] J. Yi, Y. Xie, B. Zhu, E. Kiciman, G. Sun, X. Xie, and F. Wu, “Benchmarking and defending against indirect prompt injection attacks on large language models,”

in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '25, Association for Computing Machinery, 2025.

- [16] F. Jiang, Z. Xu, L. Niu, B. Wang, J. Jia, B. Li, and R. Poovendran, “Identifying and mitigating vulnerabilities in llm-integrated applications,” 2023.